

รายงานการทดลอง

Lab: ใช้ PID ในการควบคุม Gimbal

(เลี้ยงและยิงอินฟราเรดไปยังเป้าหมาย 3 เป้า ด้วยการควบคุมแบบ PID)

[แทรกชื่อ-รหัสนักศึกษา / รายชื่อสมาชิกกลุ่ม / วันที่ทำการทดลอง]

1. วัตถุประสงค์

- ออกแบบและเขียนโปรแกรมควบคุม Gimbal ของหุ่นยนต์ RoboMaster EP ด้วยตัวควบคุมแบบ PID โดยใช้ข้อมูลตำแหน่งเป้าหมาย (Vision Marker) จากกล้องเป็นสัญญาณป้อนกลับ (feedback)
- ควบคุมให้ปืนยิงอินฟราเรด (IR) ไปยังเป้าหมาย 3 เป้า เรียงลำดับจากซ้ายสุด ไปขวาสุด แล้ววนกลับมายิงเป้าซ้ายสุดอีกครั้ง (ลำดับ 1 -> 2 -> 3 -> 1) ให้เร็วและแม่นยำที่สุด
- ศึกษาและบันทึกกระบวนการปรับจูนค่าพารามิเตอร์ PID (K_p , K_i , K_d) พร้อมปัญหาที่พบระหว่างการทดลองจริง และวิธีการแก้ไข
- บันทึกข้อมูล time response ของมุม Gimbal เพื่อนำมาวิเคราะห์และแนบในรายงาน

2. อุปกรณ์และการตั้งค่าการทดลอง

- หุ่นยนต์ DJI RoboMaster EP เชื่อมต่อผ่านโหมด Wi-Fi Direct (AP mode)
- เป้าหมาย 3 เป้า ขนาด 17 x 18 เซนติเมตร (กว้าง x สูง) เท่ากันทั้ง 3 แผ่น
- ระยะห่างระหว่างเป้าที่อยู่ติดกัน 0.6 เมตร
- ระยะห่างระหว่างหุ่นยนต์กับเป้ากลาง 1 เมตร
- ใช้กระสุนอินฟราเรด (IR) ในการยิง เพื่อความปลอดภัยและไม่เลอะเทอะ
- ตัวหุ่นยนต์ (chassis) อยู่หนึ่งตลอดการทดลอง เคลื่อนที่เฉพาะส่วน Gimbal (pitch/yaw) เท่านั้น

[แทรกภาพถ่ายการจัดวางสนามทดลองจริง (หุ่นยนต์ + เป้าทั้ง 3 + ระยะที่วัดได้)]

3. หลักการออกแบบระบบ

3.1 โครงสร้างแบบ State Machine แยกเฟส

หากปล่อยให้สัญญาณจาก Vision (ที่ทำงานแบบ event-driven ผ่าน callback) ป้อนเข้า PID และสั่ง `drive_speed()` ควบคู่ไปกับคำสั่งเคลื่อนที่แบบ open-loop (`moveto()`) พร้อมกันตลอดเวลา จะเกิดปัญหาสองคำสั่งแย่งกันควบคุมมอเตอร์ตัวเดียวกัน ทำให้ Gimbal สั่น จึงออกแบบโปรแกรมเป็น

State Machine แยก 3 เฟสต่อการยิง 1 เป้าอย่างชัดเจน ในแต่ละช่วงเวลามีเพียงคำสั่งเดียวที่ควบคุม Gimbal:

- SLEW — เคลื่อนที่แบบหยาบ (open-loop) ไปยังตำแหน่งมุมโดยประมาณของเป้าถัดไป ด้วยคำสั่ง `moveto()` โดยไม่สนใจข้อมูลจาก Vision เลยในช่วงนี้
- AIM — เปิดใช้ตัวควบคุม PID โดยรับตำแหน่ง Marker (x, y) จากกล้องเป็น feedback คำนวณ error เทียบกับกึ่งกลางเฟรม แล้วสั่ง `drive_speed()` ปรับมุมจนกว่า error จะนิ่งอยู่ในโทเลอแรนซ์ที่กำหนด ต่อเนื่องเป็นระยะเวลาหนึ่ง (Dwell Time) เพื่อป้องกันการยิงขณะเพิงแกว่งผ่านจุดกึ่งกลางเฉยๆ
- FIRE — หยุดคำสั่ง PID (`drive_speed = 0`) ก่อน แล้วจึงสั่งยิงกระสุน IR เพื่อไม่ให้ Gimbal ยังเคลื่อนที่ค้างอยู่ต่อนิ่ง

3.2 การเลือก Marker เป้าหมายด้วยตำแหน่ง แทนการอิงชื่อ/รูปสัญลักษณ์

เนื่องจากชุดเป้าจริงที่ใช้มีสัญลักษณ์ซ้ำกัน (ป้ายรูปหัวใจ 2 แผ่น และป้ายเครื่องหมาย ? อีก 1 แผ่น) การอ้างอิงว่า "เป้านี้คือเป้าไหน" จากค่า info ที่ Vision ส่งกลับมาจึงทำไม่ได้ (มีป้ายซ้ำชื่อกัน) ระบบจึงเลือกระบุเป้าหมายจาก "ตำแหน่งในเฟรมภาพ" แทน โดยเลือก Marker ที่อยู่ใกล้ตำแหน่งอ้างอิงที่กำลังติดตามอยู่มากที่สุด

3.3 การจดจำตำแหน่งมุมของเป้าที่เคยยิงสำเร็จ (Learned Angle)

เมื่อยิงเป้าใดสำเร็จแล้ว ระบบจะบันทึกมุม yaw/pitch ของ Gimbal ณ ขณะนั้นไว้กับเป้านั้น เมื่อวนกลับมายิงเป้าเดิมอีกครั้ง (เป้าซ้ายสุดในรอบที่สอง) ระบบจะสั่ง SLEW ไปยังมุมที่จำไว้โดยตรง แทนที่จะประมาณค่าจากเรขาคณิตใหม่ ทำให้เข้าใกล้เป้าหมายได้แม่นยำและรวดเร็วขึ้นในการเยือนซ้ำ

4. พารามิเตอร์ควบคุมและกระบวนการปรับจูน PID

สมการควบคุมของแต่ละแกน (yaw และ pitch) ใช้ error ที่เป็นสัดส่วนของเฟรมภาพ ($\text{error} = \text{ตำแหน่ง Marker ในแกนนั้น} - 0.5$) เป็นอินพุต และคำนวณผลลัพธ์เป็นความเร็วเชิงมุม (deg/s) ป้อนเข้าคำสั่ง `drive_speed()` ของ Gimbal โดยผ่านกระบวนการปรับจูนหลายรอบดังนี้

รอบที่ 1 — เริ่มจาก P อย่างเดียว

ตั้งค่าเริ่มต้น $K_p = 80$ (ทั้งแกน yaw และ pitch), $K_i = K_d = 0$ เพื่อดูพฤติกรรมพื้นฐานของระบบก่อน

ผลที่พบ: ระบบเคลื่อนที่เข้าหาเป้าหมายได้ แต่เกิด Steady-State Error ค้างอยู่ที่ค่าคงที่ค่าหนึ่ง

(สังเกตจากค่า error ในข้อมูล log หยุดนิ่งไม่ไหลเข้าใกล้ศูนย์ต่อ ทั้งที่ยังไม่ครบเกณฑ์ที่ตั้งไว้)

ซึ่งเป็นพฤติกรรมทั่วไปของตัวควบคุมแบบ P ล้วนเมื่อระบบมีแรงเสียดทาน/ความหน่วงจริงในโลก

รอบที่ 2 — เพิ่ม Integral และ Derivative

เพิ่ม $K_i = 20$ เพื่อไล่ Steady-State Error ที่ค้างอยู่ให้หมดไป และเพิ่ม $K_d = 6$ ควบคู่กัน

เพื่อหวังไม่ให้ผลจาก Integral ทำให้ระบบแกว่งเกิน (Overshoot)

ซึ่งเป็นแนวทางปรับจูนแบบมาตรฐาน (P สำหรับความเร็วตอบสนอง, I สำหรับกำจัด steady-state error, D สำหรับหน่วงการแกว่งที่ I อาจทำให้เกิดขึ้น)

รอบที่ 3 — พบว่าเกณฑ์ยอมรับ (Tolerance) หลวมเกินไปเมื่อเทียบกับขนาดเป้าหมายจริง

เกณฑ์เริ่มต้นตั้งไว้ที่ 3% ของเฟรมภาพ เมื่อคำนวณเทียบเป็นระยะจริงโดยใช้ขนาดป้าย (17x18 ซม.)

เทียบกับขนาด Marker ที่กล้องเห็นจริง พบว่า 3% ที่ระยะ 1 เมตร เทียบเท่าระยะเอียงได้ถึงประมาณ 13 เซนติเมตร ซึ่งกว้างกว่าครึ่งหนึ่งของความกว้างป้ายเอง (8.5 ซม.)

ทำให้บางนัดยิงไปเกือบขอบป้ายโดยยังผ่านเกณฑ์ จึงปรับลดเกณฑ์ลงเหลือ 1.5% (เทียบเท่าประมาณ 6-7 ซม.)

รอบที่ 4 — ปัญหา Marker ของเป้าข้างเคียงหลุดเข้ามาในเฟรมเดียวกัน

เนื่องจากกล้องของหุ่นยนต์มีมุมมองกว้างมาก ขณะเล็งเป้าหนึ่ง (เช่น เป้าซ้ายสุด)

กล้องยังสามารถเห็นเป้าข้างเคียง (เป้ากลาง) ปรากฏอยู่ในเฟรมเดียวกันได้เป็นบางจังหวะ เมื่อ Marker ของเป้าจริงหลุดหายไปจากเฟรมชั่วขณะ (เช่น จากการสั่นของภาพขณะ Gimbal กำลังเคลื่อนที่)

ระบบเลือก Marker ที่ใกล้กึ่งกลางเฟรมที่สุดโดยไม่มีการกรอง จึงเผลอไปเลือก Marker

ของเป้าข้างเคียงมาคำนวณ error แทน ทำให้เกิด error กระโดดขนาดใหญ่แบบฉับพลัน

และเมื่อรวมกับพจน์ Derivative ที่เพิ่งเพิ่มเข้าไป ยิ่งทำให้เกิดคำสั่งความเร็วสูงผิดปกติ Gimbal จึงสั้นอย่างรุนแรงก่อนยิง

วิธีแก้ไข: ปรับตรรกะการเลือก Marker ให้เลือกจาก "ตำแหน่งล่าสุดที่ติดตามอยู่จริง"

แทนกึ่งกลางเฟรมตายตัว และกำหนดระยะกระโดดสูงสุดที่ยอมรับได้ (Max Jump) หาก Marker

ที่ใกล้ที่สุดยังอยู่ไกลจากตำแหน่งล่าสุดเกินกำหนด จะถือว่าเป็น Marker ของเป้าอื่น

และปฏิเสธข้อมูลเฟรมนั้นไป (ถือว่าเป้าหลุดเฟรมชั่วคราว) วิธีนี้ช่วยตัดปัญหาการสั่น จากการเลือก

Marker ผิดตัวได้อย่างสมบูรณ์

รอบที่ 5 — ทดลองบีบเกณฑ์ให้แน่นเกินขีดจำกัดจริงของกล้อง

ทดลองลดเกณฑ์ลงถึง 0.2% เพื่อหวังผลความแม่นยำที่สูงขึ้น แต่พบว่าระบบไม่สามารถลู่เข้าค่านี้ได้จริง

เนื่องจากตำแหน่ง Marker ที่กล้องอ่านได้มีความละเอียดจำกัดตามจำนวนพิกเซลของภาพ (Pixel

Quantization) ผลคือระบบใช้เวลาจนครบเวลาสูงสุดที่กำหนด (Timeout) แล้วอิงตามสภาพที่ดีที่สุด

ณ ขณะนั้นแทนที่จะเป็นผลจาก การลู่เข้าจริงของตัวควบคุม

ทำให้เวลารวมทั้งภารกิจช้าลงจากเดิมหลายเท่า โดยไม่ได้ความแม่นยำที่ดีขึ้นอย่างมีนัยสำคัญ

(บางนัดยังคลาดเคลื่อนมากกว่าตอนใช้เกณฑ์ที่ผ่อนกว่าเดิมด้วยซ้ำ

เนื่องจากยังโดยใช้ข้อมูลที่ค้างอยู่ตอนใกล้หมดเวลา)

ค่าพารามิเตอร์สุดท้ายที่ใช้

พารามิเตอร์	ค่าที่ใช้	หมายเหตุ
Kp (yaw / pitch)	80.0	อัตราขยายตามสัดส่วน error
Ki (yaw / pitch)	20.0	กำจัด steady-state error
Kd (yaw / pitch)	6.0	หน่วงการแกว่งจาก I
AIM_TOLERANCE	0.005 (0.5% ของเฟรม)	เทียบเท่า ~2 ซม. ที่ระยะ 1 ม.
AIM_DWELL_S	0.25 วินาที	ต้องนิ่งในเกณฑ์ต่อเนื่องก่อนยิง
AIM_TIMEOUT_S	3.5 วินาที	กันโปรแกรมค้างหากเล็งไม่นิ่ง
MAX_MARKER_JUMP	0.10 (10% ของเฟรม)	กันเลือก Marker ของเป้าข้างเคียงผิดตัว

5. ผลการทดลอง

5.1 ความแม่นยำในการยิง (วัดจากค่าที่บันทึกได้จริง)

ความคลาดเคลื่อนคำนวณจากตำแหน่ง Marker ในเฟรมภาพขณะยิง เทียบสัดส่วนกับขนาดจริงของป้าย (17x18 ซม.) โดยไม่ต้องอาศัยค่าองศามุมมองกล้อง (Field of View) ที่ไม่ทราบแน่ชัด

นัดที่	เป้าหมาย	เยื้องแนวนอน (x)	เยื้องแนวตั้ง (y)	เวลาที่ใช้เล็ง
1	ซ้ายสุด (LEFT)	-1.0 ซม.	0.0 ซม.	2.70 วินาที
2	กลาง (MID)	-1.2 ซม.	0.0 ซม.	2.05 วินาที
3	ขวาสุด (RIGHT)	+1.6 ซม.	-0.3 ซม.	1.17 วินาที
4	ซ้ายสุด รอบ 2 (LEFT)	+1.0 ซม.	-0.7 ซม.	1.59 วินาที

เวลารวมทั้งภารกิจ (ตั้งแต่เริ่มเคลื่อนที่จนยิงนัดสุดท้าย): 7.52 วินาที

ทุกนัดยิงเข้าเป้าโดยเยื้องจากจุดกึ่งกลางไม่เกิน 1.6 เซนติเมตร (เทียบกับครึ่งความกว้างป้ายที่ 8.5 เซนติเมตร) และทุกเป้าลู่วัดเข้าเกณฑ์ที่ตั้งไว้ได้เองภายในเวลาไม่เกิน 2.7 วินาที

โดยไม่มีนัดใดต้องอาศัยการยิงแบบ Timeout

[แทรกกราฟ Time Response ของมุม yaw (จาก plot_response.py)]

[แทรกกราฟ Time Response ของมุม pitch (จาก plot_response.py)]

5.2 ข้อจำกัดทางกายภาพของระบบ

จากข้อมูลที่บันทึกไว้ พบว่าค่าตำแหน่ง Marker ในบางช่วงมีค่าคงที่ซ้ำกันหลายครั้งติดต่อกัน ห่างกันเป็นขั้นบันไดขนาดประมาณ 1/1280 ของความกว้างเฟรม ซึ่งตรงกับความละเอียดระดับ 1 พิกเซลของภาพ แสดงให้เห็นว่าระบบเล็งได้ละเอียดถึงระดับที่กล้องสามารถแยกแยะความแตกต่างได้จริง (Sub-pixel) แล้ว การปรับจูนเพิ่มเติมโดยไม่เปลี่ยนฮาร์ดแวร์ (เช่น เพิ่มความละเอียดกล้อง หรือลดระยะห่างจากเป้า) จึงไม่น่าจะช่วยเพิ่มความแม่นยำได้อีกอย่างมีนัยสำคัญ

[แทรกภาพหน้าจอ/ภาพนิ่งจากคลิปวิดีโอ ขณะยิงแต่ละเป้า (4 ภาพ)]

6. สรุปผลการทดลอง

- ระบบควบคุม Gimbal ด้วย PID ที่ออกแบบสามารถยิงเป้าหมายทั้ง 3 เป้าตามลำดับที่กำหนด (1 -> 2 -> 3 -> 1) ได้สำเร็จ ด้วยความแม่นยำเนื่องจากกึ่งกลางไม่เกิน 1.6 เซนติเมตร และเวลารวม 7.52 วินาที
- การแยกการทำงานเป็น State Machine (SLEW / AIM / FIRE)
ช่วยป้องกันปัญหาการแกว่งจากคำสั่งเคลื่อนที่สองชนิด แย่งกันควบคุม Gimbal ได้อย่างมีประสิทธิภาพ
- การเลือก Marker ด้วยตำแหน่งอ้างอิงล่าสุด (แทนกึ่งกลางเฟรมตายตัว)
ร่วมกับการจำกัดระยะกระโดดสูงสุด
เป็นแนวทางที่จำเป็นเมื่อกล้องมีมุมมองกว้างพอที่จะเห็นเป้าหมายหลายเป้าพร้อมกัน
- การปรับจูน PID ควรอ้างอิงกับขนาดจริงของเป้าหมายและระยะทดลอง ไม่ควรตั้งเกณฑ์ความแม่นยำ (Tolerance) แน่นเกินไปจนความละเอียดจริงของกล้อง เพราะจะทำให้ระบบไม่ลู่เข้าค่าจริง กลายเป็นการรอจน Timeout แทน ซึ่งทำให้เสียเวลาโดยไม่ได้ความแม่นยำที่ดีขึ้น